

Módulo 2: Exploración del Conjunto de Datos

Material de Evaluación Práctica y Ejercicios Resueltos

Este documento contiene 10 ejercicios diseñados para evaluar las capacidades cognitivas en la fase de Análisis Exploratorio de Datos (EDA).

Guía de Selección de Gráficos en Ciencia de Datos e IA

La elección del gráfico correcto es fundamental para una comunicación efectiva y un análisis riguroso. A continuación, se detallan los gráficos más habituales según el objetivo del análisis:

1. Comparación entre Categorías

- **Gráfico de Barras (Bar Chart):** Ideal para comparar magnitudes entre diferentes grupos categóricos (ej. ventas por región).
- **Gráfico de Barras Agrupadas/Apiladas:** Útil cuando queremos comparar subcategorías dentro de categorías principales.

```
# Ejemplo rápido: Comparación de categorías
sns.countplot(x=['A', 'B', 'A', 'C', 'B', 'A'], palette='pastel')
```

2. Distribución de una Variable

- **Histograma:** Permite visualizar la forma de la distribución de una variable numérica (frecuencia).
- **KDE (Kernel Density Estimate):** Una versión suavizada del histograma que ayuda a identificar picos y valles de probabilidad.
- **Boxplot (Diagrama de Caja):** Fundamental para identificar la mediana, los cuartiles y, sobre todo, la presencia de **valores atípicos (outliers)**.

```
# Ejemplo rápido: Distribución (Boxplot)
sns.boxplot(x=[10, 12, 12, 13, 14, 15, 25], color='cyan')
```

3. Relación entre Variables

- **Gráfico de Dispersión (Scatter Plot):** El estándar para visualizar la relación entre dos variables numéricas. Permite identificar correlaciones, clusters y tendencias.
- **Mapa de Calor (Heatmap):** Utilizado frecuentemente para visualizar matrices de correlación entre múltiples variables numéricas.

```
# Ejemplo rápido: Relación (Scatter Plot)
sns.scatterplot(x=[1, 2, 3, 4], y=[10, 11, 12, 15])
```

4. Evolución Temporal

- **Gráfico de Líneas (Line Chart):** La mejor opción para representar datos secuenciales en el tiempo (series temporales), permitiendo observar tendencias y estacionalidad.

```
# Ejemplo rápido: Evolución Temporal
plt.plot([1, 2, 3, 4], [10, 20, 15, 25], marker='o')
```

Criterios de Selección

Si quieres analizar...	Tipo de dato principal	Gráfico recomendado
Frecuencia de categorías	Catégorico	Barras (Countplot)
Distribución/Forma	Numérico continuo	Histograma / KDE
Outliers y Cuartiles	Numérico	Boxplot
Relación entre 2 variables	Numérico vs Numérico	Scatter Plot
Tendencia en el tiempo	Fecha/Tiempo vs Numérico	Gráfico de Líneas
Correlaciones múltiples	Matriz numérica	Heatmap

Identificación y Gestión de Valores Atípicos (Outliers)

Un **valor atípico** es una observación que se aleja significativamente del resto de los datos en una muestra. Identificarlos es un paso crítico en la fase de exploración.

¿Por qué es importante gestionarlos?

1. **Sesgo en Estadísticos:** Los outliers pueden distorsionar gravemente la media y la desviación estándar, dando una imagen falsa de la tendencia central.
2. **Impacto en Modelos de IA:** Muchos algoritmos de Machine Learning (como la Regresión Lineal) son muy sensibles a estos valores, lo que puede resultar en modelos con poco poder predictivo.
3. **Detección de Errores o Fraude:** A veces, un outlier es simplemente un error de entrada de datos; otras veces, puede representar un evento legítimo pero inusual (como una transacción fraudulenta).

Técnicas de Identificación

1. Técnicas Visuales

- **Boxplots:** Son la herramienta más rápida. Cualquier punto que caiga fuera de los "bigotes" del diagrama es considerado un candidato a outlier.
- **Scatter Plots:** Útiles para detectar outliers multivariados (datos que son normales en una variable pero inusuales cuando se combinan con otra).

2. Técnicas Estadísticas

- **Z-Score:** Mide a cuántas desviaciones estándar se encuentra un punto de la media. Típicamente, un Z-score mayor a 3 (o menor a -3) se considera atípico.
 - **Rango Inter cuartilico (IQR):** Es el método más robusto. Define límites basados en los cuartiles:
 - \$Límite Inferior = Q1 - 1.5 \times IQR\$
 - \$Límite Superior = Q3 + 1.5 \times IQR\$
-

Ejercicio 1 - Visualización Categórica en E-commerce

Objetivo: Aplicar el tipo de gráfico adecuado para representar frecuencias de categorías discretas.

Enunciado del Reto: Una tienda online desea entender qué categorías de productos son las más populares. Dispones de una lista de 500 transacciones con las categorías: 'Electrónica', 'Hogar', 'Moda', 'Libros' y 'Deportes'. Genera un gráfico que permita visualizar rápidamente el volumen de ventas por categoría.

Solución en Python:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Configuración de estilo para un acabado profesional
sns.set_theme(style="whitegrid")

# 1. Generación de datos sintéticos
categorias = ['Electrónica', 'Hogar', 'Moda', 'Libros', 'Deportes']
datos = np.random.choice(categorias, size=500, p=[0.3, 0.2, 0.25, 0.15, 0.1])
df = pd.DataFrame({'Categoría': datos})

# 2. Visualización
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Categoría', palette='viridis',
order=df['Categoría'].value_counts().index)
plt.title('Distribución de Ventas por Categoría de Producto', fontsize=14)
plt.xlabel('Categoría', fontsize=12)
plt.ylabel('Número de Ventas', fontsize=12)
plt.show()
```

Justificación: El alumno demuestra su capacidad de **Aplicar** conocimientos al elegir un `countplot` (gráfico de barras de frecuencia), que es la herramienta estándar para comparar magnitudes entre grupos categóricos.

Ejercicio 2 - Análisis de Tendencias Temporales

Objetivo: Aplicar gráficos de líneas para identificar patrones secuenciales en el tiempo.

Enunciado del Reto: Una startup tecnológica quiere analizar el crecimiento de usuarios registrados durante el primer año. Genera datos mensuales y crea una visualización que muestre si existe una tendencia de crecimiento ascendente.

Solución en Python:

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Generación de datos sintéticos (12 meses)
meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul', 'Ago', 'Sep', 'Oct',
'Nov', 'Dic']
usuarios = [120, 150, 180, 240, 310, 350, 420, 500, 610, 750, 920, 1100]

# 2. Creación del DataFrame
df_usuarios = pd.DataFrame({'Mes': meses, 'Usuarios': usuarios})

# 3. Visualización
plt.figure(figsize=(10, 5))
plt.plot(df_usuarios['Mes'], df_usuarios['Usuarios'], marker='o', linestyle='--',
color='b', linewidth=2)
plt.fill_between(df_usuarios['Mes'], df_usuarios['Usuarios'], color='skyblue',
alpha=0.3)
plt.title('Crecimiento Mensual de Usuarios Registrados', fontsize=14)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

Justificación: Demuestra la capacidad de **Aplicar** la visualización de series temporales, comprendiendo que la continuidad de la línea facilita la detección de tendencias y estacionalidad.

Ejercicio 3 - Identificación de Outliers en Tiempos de Entrega

Objetivo: Analizar la dispersión de datos y detectar anomalías visuales.

Enunciado del Reto: Una empresa de logística sospecha que algunos repartidores tienen tiempos de entrega inusualmente altos. Analiza los tiempos de entrega (en minutos) de 200 pedidos y utiliza un gráfico de caja (boxplot) para identificar visualmente si existen valores atípicos.

Solución en Python:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# 1. Generación de datos (Tiempos normales + algunos outliers)
tiempos = np.concatenate([np.random.normal(30, 5, 190), [120, 135, 150, 5, 2]])
df_logistica = pd.DataFrame({'Minutos': tiempos})

# 2. Visualización
plt.figure(figsize=(8, 6))
sns.boxplot(x=df_logistica['Minutos'], color='lightcoral')
plt.title('Análisis de Outliers en Tiempos de Entrega', fontsize=14)
```

```
plt.xlabel('Tiempo de Entrega (Minutos)')
plt.show()
```

Justificación: El uso del boxplot permite al alumno **Analizar** la estructura de los datos (cuartiles, mediana) y reconocer los puntos fuera de los "bigotes" como anomalías que requieren investigación.

Ejercicio 4 - Optimización de Tipos de Datos (Eficiencia de Memoria)

Objetivo: Analizar los factores que influyen en la elección del tipo de dato para mejorar el rendimiento.

Enunciado del Reto: Trabajas con un conjunto de datos masivo. Tienes una columna 'ID_Empleado' (enteros pequeños) y 'Salario' (enteros). Actualmente se cargan como `int64`. Analiza el consumo de memoria y propón una conversión a tipos de datos más eficientes para reducir el impacto.

Solución en Python:

```
import pandas as pd
import numpy as np

# 1. Creación de un DataFrame simulado grande
n = 100000
df = pd.DataFrame({
    'ID_Empleado': np.random.randint(1, 1000, size=n),
    'Edad': np.random.randint(18, 65, size=n),
    'Salario': np.random.randint(20000, 100000, size=n)
})

# 2. Análisis de memoria inicial
memoria_inicial = df.memory_usage(deep=True).sum() / 1024**2 # MB

# 3. Conversión de tipos (Downcasting)
df['ID_Empleado'] = df['ID_Empleado'].astype('int16')
df['Edad'] = df['Edad'].astype('int8')
df['Salario'] = df['Salario'].astype('int32')

# 4. Análisis de memoria final
memoria_final = df.memory_usage(deep=True).sum() / 1024**2 # MB

print(f"Memoria Inicial: {memoria_inicial:.2f} MB")
print(f"Memoria Final: {memoria_final:.2f} MB")
print(f"Reducción: {100 * (1 - memoria_final/memoria_inicial):.2f}%")
```

Justificación: El alumno **Analiza** la relación entre el rango de los datos y el espacio de almacenamiento, tomando decisiones técnicas que optimizan el uso de recursos computacionales.

Ejercicio 5 - Correlación en el Mercado Inmobiliario

Objetivo: Analizar la relación entre dos variables numéricas continuas.

Enunciado del Reto: En un estudio inmobiliario, se desea saber si existe una relación entre los "Metros Cuadrados" de una vivienda y su "Precio de Venta". Genera datos para 100 viviendas y crea un gráfico de dispersión con una línea de regresión para visualizar la correlación.

Solución en Python:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# 1. Generación de datos con correlación positiva
m2 = np.random.uniform(50, 250, 100)
precios = (m2 * 2500) + np.random.normal(0, 50000, 100) # Relación lineal + ruido

df_casas = pd.DataFrame({'M2': m2, 'Precio': precios})

# 2. Visualización con línea de regresión
plt.figure(figsize=(10, 6))
sns.regplot(data=df_casas, x='M2', y='Precio', scatter_kws={'alpha':0.5},
line_kws={'color':'red'})
plt.title('Correlación entre Tamaño de Vivienda y Precio', fontsize=14)
plt.xlabel('Superficie (m²)')
plt.ylabel('Precio (USD)')
plt.show()
```

Justificación: A través de `regplot`, el alumno puede **Analizar** visualmente la fuerza y dirección de la relación entre variables, un paso crítico antes de entrenar modelos de IA.

Ejercicio 6 - Detección Estadística mediante IQR

Objetivo: Analizar y filtrar datos basándose en criterios estadísticos robustos.

Enunciado del Reto: Un sensor de temperatura industrial falla ocasionalmente, registrando valores imposibles. Implementa el método estadístico del Rango Inter cuartílico (IQR) para identificar y filtrar los valores que están fuera del rango normal.

Solución en Python:

```
import pandas as pd
import numpy as np

# 1. Datos con valores atípicos severos
temps = np.concatenate([np.random.normal(70, 2, 50), [20, 150, 200]])
df_sensor = pd.DataFrame({'Temp': temps})

# 2. Cálculo de IQR
Q1 = df_sensor['Temp'].quantile(0.25)
Q3 = df_sensor['Temp'].quantile(0.75)
```

```
IQR = Q3 - Q1

limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

# 3. Identificación de outliers
outliers = df_sensor[(df_sensor['Temp'] < limite_inferior) | (df_sensor['Temp'] >
limite_superior)]
print(f"Valores atípicos detectados:\n{outliers}")

# 4. Filtrado del dataset
df_limpio = df_sensor[(df_sensor['Temp'] >= limite_inferior) & (df_sensor['Temp']
<= limite_superior)]
print(f"Tamaño original: {len(df_sensor)}, Tamaño limpio: {len(df_limpio)}")
```

Justificación: El alumno pasa de la detección visual a la lógica programática para **Analizar** anomalías, demostrando comprensión de la distribución estadística y manejo de DataFrames.

Ejercicio 7 - Análisis de Segmentos: Churn de Clientes

Objetivo: Evaluar la diferencia de comportamiento entre dos grupos mediante distribuciones.

Enunciado del Reto: Un banco quiere comparar el saldo promedio de los clientes que cancelaron su cuenta (Churn) vs. los que permanecen. Utiliza gráficos de densidad (KDE) para evaluar si el saldo es un factor diferenciador significativo.

Solución en Python:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# 1. Generación de datos sintéticos
saldo_permanecen = np.random.normal(5000, 1500, 300)
saldo_cancelan = np.random.normal(1200, 800, 100)

df_churn = pd.DataFrame({
    'Saldo': np.concatenate([saldo_permanecen, saldo_cancelan]),
    'Estado': ['Activo']*300 + ['Cancelado']*100
})

# 2. Visualización comparativa
plt.figure(figsize=(10, 6))
sns.kdeplot(data=df_churn, x='Saldo', hue='Estado', fill=True, common_norm=False,
palette='Set1')
plt.title('Distribución de Saldos por Estado del Cliente', fontsize=14)
plt.show()
```

Justificación: Al superponer distribuciones, el alumno puede **Evaluar** si una variable tiene poder discriminante, una habilidad clave para la ingeniería de características.

Ejercicio 8 - Mapa de Calor de Correlaciones Multidimensionales

Objetivo: Analizar múltiples interacciones en un dataset complejo.

Enunciado del Reto: Tienes un dataset con variables sobre salud (Edad, IMC, Presión, Azúcar, Colesterol). Genera una matriz de correlación y visualízala mediante un mapa de calor para identificar colinealidades.

Solución en Python:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# 1. Creación de datos con correlaciones forzadas
np.random.seed(42)
data = np.random.rand(100, 5)
df_salud = pd.DataFrame(data, columns=['Edad', 'IMC', 'Presion', 'Azucar',
                                       'Colesterol'])

df_salud['Colesterol'] = df_salud['Edad'] * 0.7 + np.random.rand(100) * 0.3
df_salud['IMC'] = df_salud['Azucar'] * 0.5 + np.random.rand(100) * 0.5

# 2. Matriz de correlación y Heatmap
plt.figure(figsize=(8, 6))
corr = df_salud.corr()
sns.heatmap(corr, annot=True, cmap='RdBu_r', center=0, fmt=".2f", linewidths=0.5)
plt.title('Mapa de Calor de Correlaciones Clínicas', fontsize=14)
plt.show()
```

Justificación: El alumno debe **Analizar** una matriz completa, interpretando el significado de los coeficientes y reconociendo patrones de dependencia entre múltiples variables.

Ejercicio 9 - Robustez Estadística: Media vs Mediana

Objetivo: Evaluar la idoneidad de los estadísticos descriptivos ante sesgos.

Enunciado del Reto: Calcula la media y la mediana de los salarios de una empresa donde 9 empleados ganan sueldos similares y 1 directivo gana un sueldo 40 veces superior. Determina cuál medida es más representativa de la "tendencia central" real.

Solución en Python:

```
import numpy as np
```

```
# 1. Datos: 9 empleados y 1 directivo (outlier)
salarios = [2100, 2300, 2500, 2200, 2800, 2400, 2600, 2500, 2700, 100000]

media = np.mean(salarios)
mediana = np.median(salarios)

print(f"Salarios: {salarios}")
print("-" * 30)
print(f"Media Artimética: {media:.2f}")
print(f"Mediana Estadística: {mediana:.2f}")

# Conclusión esperada:
# La media es sensible a valores extremos; la mediana es robusta.
```

Justificación: Este ejercicio obliga al alumno a **Evaluar** la calidad de su análisis descriptivo, comprendiendo que la "verdad" de los datos no siempre reside en la media aritmética.

Ejercicio 10 - Creación de un Dashboard de Diagnóstico

Objetivo: Crear un reporte visual integrado para la toma de decisiones iniciales.

Enunciado del Reto: Como analista senior, debes presentar un informe de calidad sobre un dataset de sensores ambientales. Crea un dashboard que contenga:

1. Histograma de CO2 para ver su forma.
2. Scatter plot de CO2 vs Humedad para ver su dependencia.

Solución en Python:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# 1. Datos sintéticos
n = 300
co2 = np.random.gamma(2, 20, n) + 350
humedad = 100 - (co2 * 0.1) + np.random.normal(0, 5, n)
df_clima = pd.DataFrame({'CO2': co2, 'Humedad': humedad})

# 2. Creación del Dashboard (Layout 1x2)
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Subplot 1: Histograma
sns.histplot(df_clima['CO2'], kde=True, ax=axes[0], color='salmon')
axes[0].set_title('Distribución de CO2 (Asimétrica)')

# Subplot 2: Dispersión
sns.scatterplot(data=df_clima, x='CO2', y='Humedad', alpha=0.6, ax=axes[1],
color='darkblue')
```

```
axes[1].set_title('Relación CO2 vs Humedad')

plt.tight_layout()
plt.show()
```

Justificación: El alumno alcanza el nivel de **Crear** al ser capaz de componer una visualización múltiple que sintetiza diferentes perspectivas de la información en un único artefacto comunicativo.

Actividades Propuestas (Para el Alumno)

A continuación, se presentan 10 retos adicionales para poner en práctica los conocimientos adquiridos. El alumno debe generar sus propios datos sintéticos para resolverlos.

1. **Análisis de Canales de Marketing:** Crea un gráfico que compare el número de leads generados a través de 4 canales distintos: 'SEO', 'PPC', 'Social Media' y 'Email'.
2. **Tráfico Web Semanal:** Visualiza la tendencia de visitas diarias a un sitio web durante una semana completa. ¿Qué día se observa el mayor pico?
3. **Calidad en Manufactura:** En una línea de producción de piezas metálicas, se mide el grosor de 150 unidades. Utiliza un boxplot para determinar si hay piezas defectuosas (outliers).
4. **Optimización de Dataset IoT:** Tienes un dataset con 500,000 registros de temperatura (float64) y estado del sensor (bool). Analiza el ahorro de memoria al convertir la temperatura a float32.
5. **Educación vs Salario:** Investiga la relación entre los "Años de Estudio" y el "Salario Mensual". Representa la correlación mediante un scatter plot con línea de tendencia.
6. **Control de Calidad Alimentaria:** Un lote de leche tiene mediciones de pH. Implementa el método IQR para filtrar mediciones erróneas que podrían indicar fallos en los sensores.
7. **Rendimiento de Exámenes:** Compara la distribución de notas de dos grupos de alumnos (Grupo A y Grupo B) utilizando gráficos KDE superpuestos.
8. **Variables Meteorológicas:** Crea un mapa de calor que muestre la correlación entre: Temperatura, Humedad, Velocidad del Viento, Presión Atmosférica y Radiación Solar.
9. **Precios Inmobiliarios Sesgados:** Calcula media y mediana de los precios de una calle donde 5 casas valen 200k y una mansión vale 5M. Justifica cuál usarías para una tasación justa.
10. **Dashboard de Ventas Regional:** Diseña un dashboard con dos paneles:
 - Gráfico de barras de ventas por región.
 - Histograma de la cantidad de artículos comprados por cliente.